UNIVERSIDAD POLITÉCNICA DE CHIAPAS

Ingeniería en Tecnologías de la Información e Innovación Digital

Bases de Datos en la Nube · 2026

Contención de CPU y memoria en agregaciones JSON

MongoDB frente a PostgreSQL JSONB

REPORTE ACADÉMICO PARCIAL
2 de 108 repeticiones previstas con resultados definitivos elegibles.
La matriz completa **no** se ejecutó.

Fecha de corte:	1 de noviembre de 2026
Grupo / autores:	Por completar por el estudiante
Motores:	MongoDB 9.0.4 · PostgreSQL 18.6
Dataset:	500,000 documentos JSON anidados

Documento construido con resultados confirmados. No contiene datos simulados.

Resumen ejecutivo

Este informe documenta un **avance parcial** del experimento prescrito para comparar agregaciones sobre documentos JSON anidados bajo límites de CPU y memoria. El protocolo contempla tres perfiles, tres niveles de concurrencia, dos consultas y tres repeticiones por celda, para un total de **108 repeticiones**. Al cierre documentado se dispone de **dos resultados definitivos elegibles**: Q1 con concurrencia 1 y perfil A, uno por motor. Se incluye una comparación descriptiva, no inferencial, y se identifican expresamente los escenarios pendientes.

En la ejecución observada, MongoDB obtuvo **0.5467 QPS** y PostgreSQL **0.4167 QPS**; las latencias p99 fueron **1902.25 ms** y **2800.77 ms**, respectivamente. En cambio, el RSS máximo observado de MongoDB (1026.65 MiB) superó al de PostgreSQL (108.22 MiB). Los resultados no respaldan generalizaciones a Q2 ni a concurrencia 10/50, perfiles B/C o variabilidad entre repeticiones.

Índice

1. Especificaciones del host y del entorno Docker **2**

1.1. Plataforma y controles de recursos 2

1.2. Indexación y equivalencia de motores 2

2. Método de medición y verificación del baseline **2**

2.1. Dataset y cargas de trabajo 2

2.2. Matriz y ventana temporal 3

2.3. Validación y cambios de implementación 3

3. Resultados consolidados disponibles **3**

3.1. Observaciones definitivas: Q1, perfil A, concurrencia 1 3

3.2. Cobertura y comparabilidad 4

4. Análisis gráfico **4**

5. Planes de ejecución: MongoDB y PostgreSQL **5**

6. Interpretación: memoria, derrame y throttling **5**

7. Recomendaciones técnicas para despliegues documentales en la nube **6**

Conclusión y limitaciones **6**

A. Trazabilidad de resultados y entregables **6**

B. Referencia normativa del experimento **7**

1. Especificaciones del host y del entorno Docker

1.1 Plataforma y controles de recursos

El entorno de medición utiliza contenedores Docker en Linux y `cgroups v2`. El sistema operativo reportado durante la ejecución fue Kubuntu, con Docker Engine 29.1.3. El motor documental fue MongoDB 9.0.4 (WiredTiger, colección `orders`); el relacional, PostgreSQL 18.6 (tabla `orders` con documento `JSONB`). Estas versiones se indican según las comprobaciones de la ejecución proporcionadas por el operador.

El protocolo establece los siguientes perfiles; **solamente el perfil A tiene resultados definitivos disponibles**:

Tabla 1: Perfiles exigidos por el protocolo.

Perfil	Grado de contención	CPU	RAM (MiB)	Estado
A	Baseline	4	4096	Medido: Q1, c=1
B	Media	2	2048	Pendiente
C	Severa	1	1024	Pendiente

La telemetría prevista considera `cAdvisor`, `Node Exporter` y `Prometheus` con intervalo de captura de 1 s. Los campos exportados comprenden throughput, percentiles de latencia, tiempo de throttling, RSS máximo, indicador de derrame a disco y estado. La ficha del host (modelo exacto de CPU, RAM efectiva, kernel, almacenamiento y versión de Compose) **requiere cotejo** con el manifiesto real de ejecución antes de su entrega como dato verificado; no se deduce a partir del límite de los contenedores.

1.2 Indexación y equivalencia de motores

La configuración comprende índices secundarios sobre `timestamp` y `order_id` en MongoDB, además de clave primaria sobre `id` e índice B-Tree de expresión sobre `data->>timestamp` en PostgreSQL. Existe una salvedad: Q2 compara una conversión a `timestamptz`, por lo que el índice textual no demuestra por sí mismo compatibilidad con el predicado ni uso efectivo por el planificador. La aceptación operativa de esa condición **no equivale a aprobación docente**.

2. Método de medición y verificación del baseline

2.1 Dataset y cargas de trabajo

El dataset de referencia consta de 500,000 órdenes JSON de tres niveles, con promedio aproximado de tres ítems por orden. Contiene campos de pedido, tiempo, cliente y ubicación, ítems y precio, pago y metadatos. El archivo fuente usado fue `data/orders-42.jsonl`; SHA-256 comunicado para la fuente: `9fecf656a02cf6d369517ae4d59f47072e4de4025ae9db6280611347fb557afb`. La comparación de datos entre motores había terminado con estado PASS en la preparación, según evidencia previa del proyecto; el código del generador y la prueba completa permanecen en el repositorio del operador.

Q1: Unwind, Filter & Grouping. Desanida `items`, filtra `price > 50`, calcula qty × price y agrega suma, promedio y cantidad por país y categoría. En PostgreSQL utiliza `jsonb_array_elements`; en MongoDB, `$unwind`, `$match` y `$group`.

Q2: Massive Sort & Top-K. Filtra órdenes completadas desde el 1 de enero de 2026, ordena por monto final descendente y devuelve los 1000 primeros registros. Variantes previstas: MongoDB `allowDiskUse=false/true`; PostgreSQL `work_mem=4/64MB`. **No hay mediciones definitivas de Q2** en esta versión.

2.2 Matriz y ventana temporal

La matriz base consta de 2 motores ×3 perfiles ×2 consultas ×3 concurrencias (1, 10, 50) = 36 celdas. Con 3 repeticiones por celda se requieren 108 repeticiones. Cada una prescribe: vaciado de caché del sistema aprobado, **60 s** de calentamiento, **300 s** de medición y **60 s** de enfriamiento. La suma nominal de ventanas es de 756 minutos (12 h 36 min), sin preflight ni operaciones auxiliares.

2.3 Validación y cambios de implementación

La primera medición PostgreSQL se realizó mediante preflight individual, mientras que la medición MongoDB utilizó una versión que *permite* compartir evidencias de validación entre escenarios de una invocación, sin alterar consultas, límites ni ventanas. El preflight de MongoDB fue de tipo `FULL`, con duración de **2159.565 s**. La tentativa de reutilización para el siguiente escenario (PostgreSQL) quedó **BLOCKED** por diferencias en identidad, recuento o contadores de escritura; **no** se completó ese segundo escenario en la invocación optimizada. En consecuencia, la mejora de tiempo **no fue demostrada** mediante una segunda medición. La suite local reportó **135 pruebas, sin fallos, 12 omitidas**.

3. Resultados consolidados disponibles

3.1 Observaciones definitivas: Q1, perfil A, concurrencia 1

Tabla 2: Valores observados; una repetición por motor. No representan agregación de tres repeticiones.

Métrica	MongoDB	PostgreSQL
Throughput (consultas/s)	0.5467	0.4167
Latencia p50 (ms)	1818.41	2353.83
Latencia p95 (ms)	1861.30	2562.62
Latencia p99 (ms)	1902.25	2800.77
CPU throttled (s)	0.000000	0.083895
RSS máximo (MiB)	1026.65	108.22
Derrame a disco detectado	No (0)	Sí (1)
Estado del resultado	ok	spill
Consultas completadas en 300 s	164	125
Elegible para análisis	Sí	Sí

En este par de ejecuciones, el throughput de MongoDB fue **31.2 % mayor** respecto al de PostgreSQL. Los percentiles de latencia resultaron menores en MongoDB: p50 aproximadamente 22.7 %, p95 27.4 % y p99 32.1 % menos. PostgreSQL utilizó menos RSS máximo: la relación MongoDB/PostgreSQL fue de alrededor de **9.49 veces**. Estos contrastes son **descriptivos y preliminares**; cada motor tiene una sola observación y no hay dispersión entre repeticiones disponible.

3.2 Cobertura y comparabilidad

Las dos filas proceden de *invocaciones distintas*: PostgreSQL de `definitivo-pg-q1-a-c1-20261101-v2`, MongoDB de `definitivo-q1-perfil-a-optimizado-20261101`. Ambas declaran `eligible_for_final_analysis=true` y cuentan con archivos de confirmación `commit.json`. El cambio de periodicidad del preflight debe mantenerse en la trazabilidad, por lo que no se presenta el par como una comparación pareada repetida o como una matriz consolidada completa. Las corridas piloto anteriores no forman parte de esta tabla: emplearon otras condiciones de preparación y ventanas de duración menor.

4. Análisis gráfico



Figura 1: Throughput Q1: únicamente perfil A, concurrencia 1, repetición 1.

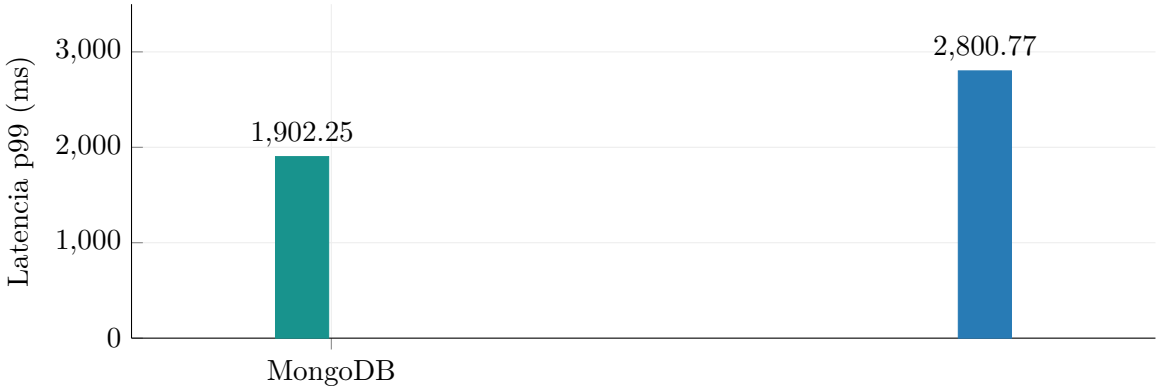


Figura 2: Latencia p99 Q1. No hay datos para construir una curva p99 frente a concurrencia 10 y 50.



Figura 3: Máximo RSS observado en las mediciones definitivas disponibles.

La relación entre throttling y QPS no puede caracterizarse con dos puntos observados en condiciones únicas. Tampoco puede generarse una *curva temporal* de RSS a partir de un solo máximo por ejecución. Esas visualizaciones se reservan para una extracción posterior de la telemetría cruda, sin interpolar ni inventar datos.

5. Planes de ejecución: MongoDB y PostgreSQL

El protocolo requiere contrastar `explain('executionStats')` de MongoDB con `EXPLAIN (ANALYZE, BUFFERS)` de PostgreSQL. **No se adjuntaron las salidas originales de estos comandos** para las ejecuciones definitivas incluidas; por honestidad metodológica no se atribuyen aquí planes, tiempos internos, cantidad de páginas ni tipos concretos de escaneo.

Tabla 3: Evidencia de plan de ejecución requerida y estado.

Motor	Evidencia esperada	Estado
MongoDB	`explain('executionStats')` de Q1, número de documentos examinados, fases y uso de memoria	Pendiente de adjuntar
PostgreSQL	`EXPLAIN (ANALYZE, BUFFERS)` de Q1, tipo de agregación, tiempos, bloques y ordenamiento	Pendiente de adjuntar

En Q1, el procesamiento desanida arreglos y agrega por país y categoría; los accesos y operaciones materiales deben verificarse en los planes reales, no inferirse de la sintaxis SQL o de la canalización por sí sola. Para Q2, el índice B-Tree textual de PostgreSQL tiene una discrepancia frente a la conversión temporal de su filtro y **no puede declararse utilizado** sin evidencia del planificador.

6. Interpretación: memoria, derrame y throttling

Memoria. MongoDB registró 1026.65 MiB de RSS máximo y PostgreSQL 108.22 MiB. La comparación se refiere a la métrica de máximo RSS recogida por cada ejecución, no al tamaño total de cachés del host ni al consumo energético. Un RSS menor no determina por sí solo un motor más eficiente: deben evaluarse también throughput, percentiles y derrame.

Derrame a disco. Para la medición de PostgreSQL se registró `disk_spill=1` y estado `spill`. MongoDB registró `disk_spill=0`, estado `ok`. Sin planes y trazas internas no puede cuantificarse el volumen del derrame ni asignarse una causa única.

CPU. El tiempo de *throttling* fue 0.083895 s en PostgreSQL y 0.000000 s en MongoDB. La magnitud observada para este perfil de 4 CPU no prueba un patrón de contención general bajo los perfiles B o C, que siguen pendientes.

Estabilidad. Las dos repeticiones enumeradas terminaron elegibles con 125 y 164 consultas completadas, respectivamente. No se extrapolan conclusiones sobre OOM o fallos a cargas con más concurrencia o menos RAM.

7. Recomendaciones técnicas para despliegues documentales en la nube

1. **Dimensionamiento por carga real.** Para Q1 y bajo el perfil A observado, MongoDB presenta mejor throughput y p99, pero mayor RSS. Una recomendación de motor necesita contrastar estos compromisos con más repeticiones y perfiles.
2. **No extrapolar Q1 a Q2.** El ordenamiento Top-K y el tratamiento de buffers son diferentes de la agregación por grupos. Deben medirse ambas variantes de `allowDiskUse` y `work_mem` antes de seleccionar una estrategia.
3. **Separar validación y medición sin perder trazabilidad.** La reutilización de preflight requiere pruebas de inmutabilidad adecuadas. En esta invocación el control detectó una diferencia y bloqueó la reutilización: debe conservarse como incidencia, no corregirse alterando evidencias.
4. **Monitoreo y alertas.** Conservar series temporales de cgroups, métricas de I/O y señalización OOM para estudiar throttling, RSS y derrame en condiciones severas.
5. **Comparaciones reproducibles.** Mantener índices, versiones, definición de consultas, política de caché y muestras de planes verificadas; analizar resultados de 3 repeticiones por celda cuando existan.

Conclusión y limitaciones

El resultado parcial de Q1 en perfil A y concurrencia 1 sugiere menor latencia y mayor tasa de consultas para MongoDB, junto con un RSS máximo sustancialmente superior. PostgreSQL mostró RSS menor, con derrame a disco detectado. **No es una conclusión estadística**: solamente se completaron 2 de las 108 repeticiones previstas, ambas con condiciones declaradas como definitivas, pero en invocaciones de preflight diferentes. No existen resultados de Q2, de perfiles B/C ni de concurrencia 10/50 para este corte. La evidencia de planes de ejecución y series temporales sigue pendiente. Ninguna cifra faltante ha sido inferida o reemplazada por datos sintéticos.

A. Trazabilidad de resultados y entregables

Las fuentes de las cifras son los `results.csv` originales de 14 columnas y los `result.json` confirmados por `commit.json`, correspondientes a:

- `definitivo-pg-q1-a-c1-20261101-v2`, Q1/PERFIL A/c1/r1, PostgreSQL.
- `definitivo-q1-perfil-a-optimizado-20261101`, Q1/PERFIL A/c1/r1, MongoDB.

La copia auxiliar de las dos filas está en `generated/measurements_partial.csv`, con esquema CSV de 14 campos. El proyecto contiene `update_results.py`, que permite regenerar métricas tras verificar la elegibilidad de los resultados y hashes del `commit.json`. **No sustituye** las carpetas originales ni un consolidado completo de la matriz.

Entregables exigidos por la guía: repositorio con Compose, generador de dataset, consultas y runner; CSV de mediciones; informe con siete secciones. Los archivos de scripts y Docker existen en el equipo del operador según su registro de ejecución; la integridad del repositorio final debe verificarse antes de la entrega.

B. Referencia normativa del experimento

Universidad Politécnica de Chiapas. *Medición de métricas en bases documentales: Contención de CPU y memoria en agregaciones JSON, MongoDB frente a PostgreSQL JSONB*. Material de la asignatura Bases de Datos en la Nube, programa IT2iD, 2026 (`playbook.pdf`, páginas 1–4).